Korotkin Gleb, Ivan Kozhanov

September 2026

Solution

0.1 General results

The developed chip is a hardware implementation of one variant of the Two Star Battle puzzle for an 11×11 matrix. Using the I and enable signals, the user enters a value of 1 (a star is present) or 0 (there is no star) for each of the 121 cells of the square. The circuit checks whether the specified regions have been filled correctly and whether the game rules have been followed (stars must not touch horizontally, vertically, or diagonally; there must also be exactly two stars in every column and row). After filling the square, the user receives the result of checking the puzzle solution. If there are no stars, the response is **EMPTY SKY**; if the entire square is filled, it is **BIG BANG**; for an incorrect answer with between 1 and 120 stars, it is **TRY AGAIN**; if the stars touch one another horizontally, vertically, or diagonally, it is **TWO NOT TOUCH**; and for a correct solution, the user receives **(* TWO STARS *)**. The correct and unique solution is shown below.

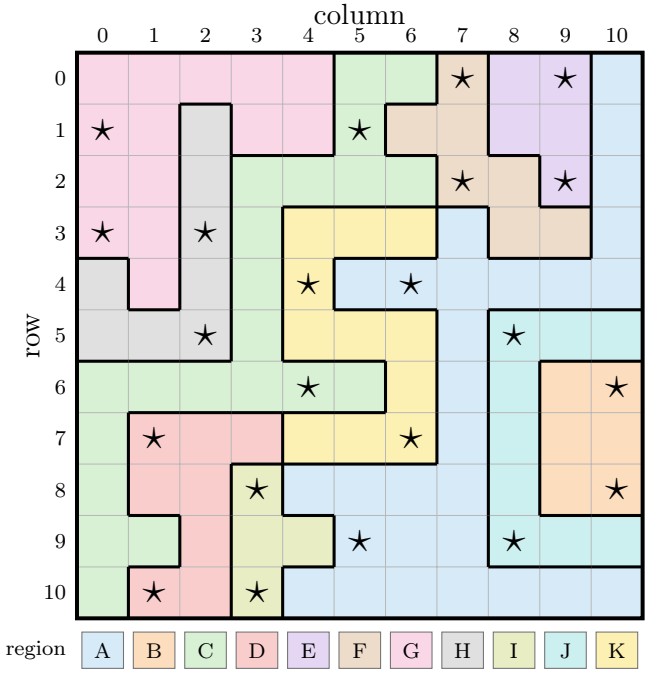


Figure 1: Unique solution of the recovered 11×11 Two Stars puzzle.

0.2 Netlist analysis

While solving the task, a tool was developed for analyzing GDS files that use the SKY130 cell library. In addition to recovering the netlist, it was also possible to divide it into modules that perform particular logical functions. The schematic of the top module is shown in the figure and attached to the report 2.

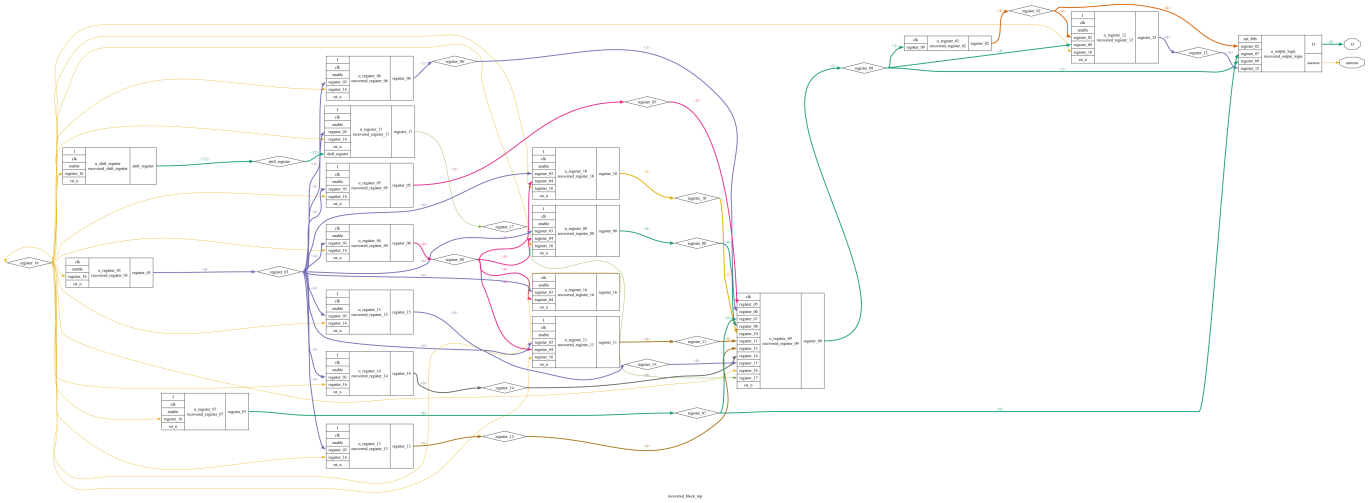


Figure 2: Yosys generated schematic

The structure shows blocks that process the serial input I according to particular rules when permitted by the enable signal. These blocks pass their states to a special block, u_register_09, which updates the computation result on every clock cycle and produces flag values that are stored in register_09. The flag values are passed to the u_output_logic module. Let us examine u_output_logic more closely, since it generates the output character and the signal indicating a correct input. Its operation depends on four variables: register_02, register_07, register_09, and register_12. We begin with register_02. Its module has one input variable, register_09. It is sufficient to write a simple testbench that iterates over its possible values in order to obtain all variants of the output signal.

```
‘timescale 1ns/1ps

module tb_02;
    recovered_register_02 dut (
        .clk(clk),
        .register_09(register_09),
        .register_02(register_02)
    );
    reg clk = 0;
    reg [2:0] register_09 = 3’b000;
    wire [3:0] register_02;

    always begin
        #5 clk = ~clk;
    end
    integer i;
    initial begin
        @(negedge clk);
        register_09 = 3’b001;
        for(i = 0; i < 16; i = i + 1) begin
            @(posedge clk);
            $display(register_02);

        end
        $finish;
    end

endmodule
```

The output values are the numbers from 0 to 15 in a mixed order. This is connected with the impossibility of directly recovering the particular register-bit arrangement intended by the developer. If the structure of the recovered code is examined more closely, it can be seen that the combinational logic resembles an adder circuit. Therefore, after restoring the bit order, the exact output values are: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15. Thus, it is a 4-bit counter that is enabled and reset according to the value of register_09[0]. A similar examination can be performed for u_register_07, which is an 8-bit counter of the input serial values (during testing, it can be seen that the output value increases by one when a high input bit is received). The remaining variable, register_12, acts as a key. During data input, u_register_12 computes a signature byte using a special algorithm. After computation over the input data stops, the block switches to output mode. For every character, the computed signature byte evolves according to the rule below; this structure is not difficult to notice in the netlist schematic.

```
function [7:0] next_key; // output mode
        input [7:0] q;
        begin
            next_key[7] = q[3] ^ q[1] ^ q[5] ^ q[7];
            next_key[6] = q[2] ^ q[4] ^ q[5];
            next_key[5] = q[4] ^ q[5] ^ q[1] ^ q[0];
            next_key[4] = q[5] ^ q[7] ^ q[6] ^ q[2];
            next_key[3] = q[4] ^ q[0] ^ q[6] ^ q[3];
            next_key[2] = q[4] ^ q[2] ^ q[3] ^ q[1];
            next_key[1] = q[3] ^ q[5] ^ q[7] ^ q[6] ^ q[0];
            next_key[0] = q[2] ^ q[1] ^ q[7];
        end
    endfunction
```

The operating mode of u_register_12 is determined by the register_16 signal, which can be replaced by the more meaningful name calculation_ready. Once the purpose of the variables is understood, it is sufficient to write a testbench that iterates over their values to obtain all possible outputs. When selecting the key, it is important to update the possible key value through the **next_key** function for every output character.

Output string	register_07	Logical count	register_09[2:0]	Initial register_12
No output	any	any	{any}0	any
EMPTY SKY	8'h00	0	{any}1	any
BIG BANG	8'd121	121	{any}1	any
TRY AGAIN	except {8'h00, 8'd121}	1...120	3'b001	any
TWO NOT TOUCH	except {8'h00, 8'd121}	1...120	3'b011	any
(* TWO STARS *)	8'd22	22	{3'b101, 3'b111}	8'h65

Table 1: Output strings

The table above shows the string outputs found while testing the u_recovered_output module and the corresponding parameter values. Testing also suggests that register_03 and register_04 are the column and row numbers, respectively. u_register_06 counts the number of stars in a row and sets a special flag to 1 if a rule violation is present, after which it passes this flag to u_register_09. The u_register_05, u_register_15, u_register_14, and u_register_13 blocks count the stars in columns 0, 1–4, 5–7, and 8–10, respectively. The u_register_08, u_register_10, and u_register_11 modules count the stars in particular regions using the 2-bit counters into which registers 08, 10, and 11 are divided: u_register_08 handles regions (A, B, C), u_register_10 handles (D, E, F, G), and u_register_11 handles (H, I, J, K), where the letters correspond to the regions in Figure 1. The

regions can also be recovered through testing because each 2-bit region counter has an enable signal that depends on the row and column numbers. By iterating over the row and column numbers, the enable signal can be read and a special symbol can be placed in the corresponding matrix cell. In this way, the region in which the number of stars must equal two is highlighted. In summary, by writing tests for individual logical modules, it is possible to infer the meaning of their operation without studying the recovered logic in great detail. This method makes it easy to recover counters, the operation of small circuits, or circuits with a small number of input signals. Various heuristics can be used to find parts of the circuit that speed up the analysis. One such approach is to search for signals that select the circuit's operating mode, or enable signals that indicate that target data has been received for processing (in this case, for example, counting a high input value when its coordinates lie within the monitored region).

Reverse tool

To solve the task, a tool was developed that uses the KLayout Python API and the SKY130 PDK. These tools make it possible to obtain a graph that describes the circuit. A module can then be assembled by using the BFS graph traversal algorithm from a vertex representing a logical element until every possible path terminates at a flip-flop:

```
1
2  module comb_module(
3      input wire [n:0] recovered_register,
4      output wire some_net
5  );
```

Recovering register groups is a separate task. One possible approach is to use various metrics, such as a shared clock and reset network, close placement, the same cell type, and so on. Based on these data, two flip-flops can be compared to determine whether they belong to the same register group. If the algorithm is started from a wire that is the input of a flip-flop, a module of the following form can be obtained:

```
1
2  module comb_module(
3      input rst,
4      input wire clk,
5      input wire [n:0] recovered_register_x,
6      output reg ff_Q
7  );
8
9  wire D;
10
11 comb_logic_between comb(
12     .input_register(recovered_register_x),
13     .output_D(D)
14 )
15
16 always @(posedge clk) begin
17     if(rst) ff_Q <= 0;
18     else ff_Q <= D;
19 end
```

The traversal may also terminate at circuit inputs such as I, enable, and so on. Also the source code will be posted on my page